

Introduktion till C – föreläsningsanteckningar.

Av: Johan Havner - Niklas Mårdby - Daniel Winckler

Lite historia.

Programspråket C skapades av en mängd personer under 1970-talet. Det började med Ken Thompson som skapade språket B under 1969-70. Språket föddes ur Martin Richards BCPL. Dennis Ritchie omvandlade B till C under åren 1971-73. Han sparade det mesta av syntaxen från B för att kunna skapa en övergång från B till C då Unix samtidigt utvecklades och skrevs i B och sedan C. Vi går här inte in på skillnaderna mellan språken då detaljerna är många och ofta kompliserade.

För en bättre och grundligare genomgång av historien hänvisar vi till "The Development of the C Language" av Dennis Ritchie (<http://cm.bell-labs.com/cm/cs/who/dmr/chist.html>). Mycket intressant läsning.

Men avslutningsvis för att komma till C, kan man nämna att första versionen av C kom 1978. Mycket har hänt sedan dess. 1983 tillsatte ANSI en grupp som skulle ta fram "en otvetydig och maskinoberoende definition av språket C", utan att ta bort något väsentligt eller förändra något viktigt. Resultatet av detta blev C som vi kommer att lära känna det.

Ett litet program.

```
#include <stdio.h>

main()
{
    printf("Hello, human!\n");
}
```

Skrivs det ovan in i en vanlig textfil som sparas med namnet test.c, där filändelsen .c är det som är lämpligt att observera, och kompileras så kommer man att få en exekverbar fil, ett program, som kommer att skriva ut följande:

```
Hello, human!
```

Varför det blir som det blir?

Raden `#include <stdio.h>` ger programmet tillgång till en hel massa standardiserade I/O-funktioner. Språket C i sig självt är inte särskilt stort, det är alla biblioteken med funktioner som man kan använda sig av som gör det så mångsidigt och användbart.

Raden `main()` deklarerar helt enkelt en funktion som heter just main, den funktion som är den första som körs i alla C-program.

Raderna med `{` och `}` gör inget annat än att de påbörjar och avslutar main.

Raden `printf("Hello, human!\n");` använder sig som synes av `printf`-funktionen, en av alla de funktioner som finns i biblioteket `stdio.h`. Den skriver ut strängen "Hello, human!\n", en sträng som dels innehåller texten "Hello, human!" men även tecknet för att lägga in en radbrytning, `'\n'`.

Hur gör man sen då?

För att kompilera filen `test.c` så skriver man, på en dator i NC eller MC:

`cc test.c` eller `gcc test.c`.

Detta ger en exekverbar fil som heter `a.out`. Vill man bestämma själv vad den ska heta, exempelvis `test`, skriver man istället:

`gcc test.c -o test`

De två olika kompilatorerna, `cc` och `gcc`, ska inte skilja sig alls mycket; ingen av de sakerna vi försökte oss på gav olika resultat.

Vill man använda andra bibliotek än standardbiblioteken så skriver man:

`gcc -Lpath -lname filename.c -o output_filename`

Där `-Lpath` och `-lname` berättar för kompilatorn var biblioteken ligger och vad de heter.

Vill man kommentera sin kod, något som man givetvis vill, så finns det två sätt:

- Antingen så använder man sig av `/*` och `*/`, allting mellan dessa båda kommer att ignoreras av kompilatorn.
- Eller så skriver man `//`, vilket får till följd att allting som står efter de båda snedstrecken på den raden kommer att ignoreras.

Variabler och aritmetik.

```
#include <stdio.h>
main()
{
    int a, b;                // Variablerna a och b deklareraras.
    a = 0;                   // a tilldelas värdet 0.
    b = 5;                   // b tilldelas värdet 5.
    while(a <= b)            // Loopa så länge a <= b.
    {
        printf("%d\t", a, b) ;
        a = a + 1;
    }
    printf("\n");
}
```

While-satsen på rad 7 är en vanlig programkonstruktion i C. En sådan fungerar i stort så här:

```
while(condition)
{
    statements
}
```

I statements så bör ju något hända med en variabel som är med i condition annars fastnar man troligtvis i loppet för evig tid.

Logiska operatorer.

De logiska operatorer som C innehåller skiljer sig inte nämnvärt, om något, från de som finns att tillgå i alla andra programspråk:

==	lika med
!=	inte lika med
<	mindre än
<=	mindre än eller lika med
>	större än
>=	större än eller lika med
&&	och
	eller

Nämnas bör att alla logiska operatorer går före ”tilldelnings=”, ett faktum som får till följd att följande två rader är syntaktiskt lika:

```
b = a == 4;  
b = (a == 4);
```

Funktionen printf.

Denna funktion skriver ut saker, i princip vad som helst. Nedan en liten lista med exempel:

Funktionsanrop:	Output:
printf("Haj!\n");	Haj!
printf("%d hajar!\n", 4);	4 hajar!
printf("%5.2f hajar!\n", 123.4567);	123.46 hajar!

Det första exemplet är självförklarande, det skriver helt enkelt ut Haj! och byter till nästa rad innan det är klart. I det andra så står det %d och sedan, till synes efter strängen, så står det en ensam fyra. Den ensamma fyran är vad printf stoppar in där det står %d. I det tredje och sista exemplet så står det %5.2f och talet som stoppas in där är 123.4567, inte riktigt samma som kommer ut. Varför inte?

F:et i printf står för format, som användare har man möjlighet att formatera inte bara vad printf ska skriva utan även **hur** det ska skrivas.

Några av de saker man kan skriva med printf är:

%d	heltal, decimalt
%xd	som %d, minst x tecken långt
%f	flyttal
%xf	flyttal, minst x tecken långt
%x.yf	flyttal, minst x tecken långt och med y decimaler

Det finns många fler %-konstellationer som känns igen, går det att tänka ut så kan nog printf lösa det.

For-satsen.

En annan väldigt vanlig progrankonstruktion i C är for-loopen. Det lilla programmet nedan skriver ut talen 0 till 5 med en tab emellan varje:

```
#include <stdio.h>
main()
{
    int a;
    for(a=0; a<5; a=a+1)
    {
        printf("%d\t", a);
    }
    printf("\n");
}
```

For-loopen som sådan har följande form i C:

```
for(int start; loop condition; loop expression)
{
    statements
}
```

En for-loop i C känns lätt igen av någon med kännedom om motsvarande konstruktion i Java, det finns dock en skillnad som är värd att peka på. I Java får man deklarerat integern man tänker använda som loopvariabel inne i själva anropet av for, det får man inte i C.

Java: `for(int a = 0; a<5; a=a+1)`

C: `int a;`
`for(a = 0; a<5; a=a+1)`

Symboliska konstanter.

En otroligt användbar sak i C är symboliska konstanter. Antag att man vill skriva ett program som räknar ut priset för inhandlandet av olika antal av tre olika varor i en affär. Rimligtvis så kommer pris1, pris2 och pris3 att återfinnas på ett flertal platser i koden, något som gör det otroligt besvärligt att ändra pris på en vara. Det är inte bara det att man måste ändra på en massa ställen, dessutom så är det lätt att missa någon enstaka förekomst av prisn. En miss som är lätt att göra men kanhända så blir den väldigt svår att upptäcka.

Hur använder man då symboliska konstanter för att slippa detta?

`#define namn ersättningstext`

Hur ser det då ut i ett program? Jo, som följer:

```
#include <stdio.h>
#define PRICE 100

main()
{
    int a;
    a = 3;
    printf("De kostar %d kronor styck.\nDu vill ha %d
stycken, det kostar %d kronor.\n", PRICE, a, a*PRICE);
}
```

Om vi hade skrivit detta program utan symboliska konstanter och i efterhand velat ändra priset, PRICE, så hade vi behövt ändra på två ställen. Tack vare att vi gjort som vi gjort så räcker det med att ändra 100 på rad två till något annat. Givetvis kan man göra allvarliga fel med detta, kompilatorn tar ju helt sonika ersättningstexten och stoppar in det som står där istället för konstantens namn på alla platser i koden som den förekommer på. Här, som på alla andra ställen när man jobbar med C, så gäller det att vara på sin vakt. Skriver vi till exempel "tyg" istället för 100 så säger kompilatorn ifrån när vi i anropet av printf försöker multiplicera en sträng med ett tal.

If-satsen.

Den tredje vanliga programkonstruktionen som vi ämnar ta upp är if-satsen. Programmet nedan visar hur den fungerar i C:

```
#include <stdio.h>
main()
{
    char a;
    printf("En siffra mellan ett och fem: ");
    a = getchar();
    if(a == '1'){
        printf("En etta.");
    }
    else if(a == '2'){
        printf("En tvåa.");
    }
    else{
        printf("Varken en etta eller en tvåa.");
    }
    printf("\n");
}
```

Programmet är enkelt. Användaren matar in en siffra mellan ett och fem. Är siffran ett så nekräftar programmet det, lika så med en tvåa. Är det ingen av de två så konstaterar programmet just det, att varken en etta eller en tvåa matades in. Givetvis så får man svaret "Varken en etta eller en tvåa." för alla tacken man matar in som inte är en etta eller en tvåa.

Formen för en if-sats:

```
if(condition1){
    statements1
}
else if(condition2){
    statements2
}
else if(condition3){
    statements3
}
else{
    statements4
}
```

Arrays.

Programmet nedan använder sig av en array av chars för att skriva ut ordet hund. Ett omständigt sätt, det medges, men det visar hur man använder arrays.

```
#include <stdio.h>
main()
{
    char a[4];
    int b;
    a[0] = 'h';
    a[1] = 'u';
    a[2] = 'n';
    a[3] = 'd';
    for(b=0; b<4; b++)
        printf("%c", a[b]);
    printf("\n");
}
```

En array i C är indexerad från 0 och fram till n-1 där n är antalet element i den. I programmet ovan har vi char a[4], en array med elementen a[0], a[1], a[2] och a[3]. Som synes så deklareras de enligt nedan:

```
type name[number of elements];
```

Java varnar om man går utanför arrayindexgränserna. I C kan man gå utanför givna indexgränser utan att programmet säger något, det läser då från minnet på ett sätt som icke är önskvärt. Vidare vad gäller skillnader mellan C och Java så är en sträng i Java en typ medans den i C är en array av chars som avslutas med nulltecknet, '\0'. Läser man in filer i C så ska man inte lagra det man läser in i chars utan i ints. Anledningen är att markören som visar att filen är slut, EOF, inte får plats i en char men i en int.

Vill man ha flerdimensionella arrays så skriver man till flera hakparentespar. Raden nedan skapar en 2 gånger 4 element stor array av chars.

```
char a[2][4];
```

Funktioner.

Funktioner är själva programmet, ett litet program består av kanske en enda funktion, ett stort program kanske innehåller några dussin funktioner. Alla funktioner i C har formen gemensamt:

```
return-type funcname(parameter declarations, if any)
{
    declarations
    statements
}
```

Exempel:

```
int add2(int n, int o)
{
    int m;
    m = n+o;
    m = m+2;
    return m;
}
```

- **return-type**
Här står det vad funktionen ska returnera, om den inte ska det så skriver man void här. Vår exempelfunktion ska returnera en int.
- **funcname**
Funktionens namn, används av andra funktioner för att köra den. Exempelfunktionen heter add2.
- **parameter declarations, if any**
De argument som funktionen tar. add2 tar två int som inuti funktionen kallas för n och o. Flera parametrar åtskiljs med kommatecken. Inga parametrar markeras med void.
- **declarations**
Variabler som funktionen använder men inte tar som argument. Vår exempelfunktion deklarerar en int, kallad för m.
- **statements**
Det som funktionen gör.
Vår exempelfunktion lägger först ihop n och o och lagrar resultatet i m. Sedan läggs m ihop med 2 och återlagras i m. Slutligen så returnerar funktionen värdet för m, alltså de båda invärdena plus 2.

Alla funktioner utom main() måste deklarerars i början av filen. Fördelen med detta är att man som programmerare ser vilka funktioner som finns i en fil utan att behöva ta sig igenom en massa kod.

Detta kallas för *function prototype*. I den behöver man inte kalla parametrarna för något, bara ange vilka typer de har.

```
#include <stdio.h>
int add2(int); ← Function prototype.
```

```
main(){
    int m;
    m = 2;
    printf("%d\n" , add2(m));
}
```

```
int add2(int add2n){
    return add2n+2;
}
```


Call by value.

När en funktion körs så skapar den alla variabler som deklarerats. De variabler som är parametrar för funktionen skapas. De får värden inkopierade från de variabler som gavs som argument när funktionen anropades.

Exempel:

```
#include <stdio.h>
int incadd(int, int);

main(){
    int n1, n2, tot;
    n1 = 5;
    n2 = 3;
    tot = incadd(n1, n2);
    printf("%d+%d=%d\n", n1, n2, tot);
}

int incadd(int x, int y){
    x = x+1;
    y = y+1;
    return x+y;
}
```

Exemplet ovan skriver ut $5+3=10$. Detta är uppenbart matematiskt fel. Dock så visar det tydligt hur *Call by value* fungerar.

När main anropar incadd så kopieras värdena för n1 och n2 in i variablerna x och y i incadd. Alla operationer som utförs av den anropade funktionen påverkar bara kopiorna av de givna variablerna, inte själva variablerna i den anropande funktionen.

I exemplet så lägger incadd på 1 på både 3 och 5. Sedan adderar den de båda delresultaten och returnerar slutligen 10. Dessvärre så skriver main efter sitt anrop av printf det felaktiga $5+3=10$ istället för det korrekta $6+4=10$.

Funktionen incadd kommer alltså inte åt variablerna i main, då hade det ju blivit en korrekt utskrift, utan bara deras värden.

Detta sätt att gå tillväga kallas för *Call by value* eftersom endast värdena för den anropande funktionens variabler skickas vidare till den anropade funktionen, inte variablerna själva.

Om man däremot ger en array som argument till en funktion så ger man därigenom funktionen full tillgång till arrayn, ingen ny array skapas och inget kopierande förekommer.

External variables.

I C kan man deklarera variabler som externa, de ligger inte in någon funktion och kan komma åt av alla funktioner. Detta är givetvis lockande, har man allting externt deklarerat så behöver man ju aldrig bry sig om vad man ger som argument till olika funktioner. Det är en farlig tanke. Lockande kanske, men farlig. Kontrollen tappas snart. När byter variabler värden? När händer vad? Vad händer? I små program kan man använda externa variabler men det är inte en vana man bör skaffa sig.

```
#include <stdio.h>
int max;

void testtext(void);

main()
{
    extern int max;
    max = 3;
    testtext();
    printf("%d\n", max);
}
void testtext(void)
{
    max = max + 2;
}
```

Programmet skriver ut 5, testtext ändrar således värdet på den externa variablen max.

Avslutande ord.

För att få en känsla för C är så klart tipset: "Sätt dig direkt med emacs och börja skriva. Kompilera och njut." Jag vågar gissa att resultatet mycket väl kan vara oväntat, då C är ett språk som är kraftfullt men oförlåtande av fel. Vänj dig vid det och se till att ha tungan i rätt mun. Prova bara att göra en for-loop från 1 till 10 som skriver ut elementen i en array som bara har 5 element...

Men med den kunskap du har nu bör du kunna skriva ganska omfattande program, så sätt igång.